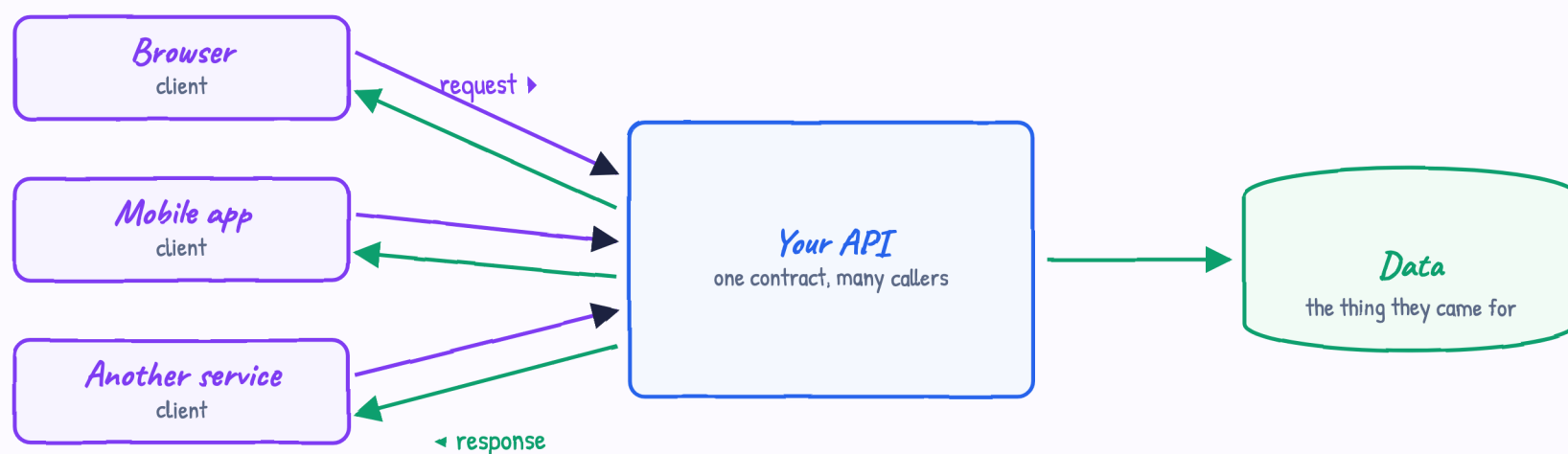




API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins



What's inside

- ✓ What an API style actually decides
- ✓ REST — resources and verbs
- ✓ GraphQL — the client picks the fields
- ✓ gRPC — a contract, not a URL
- ✓ WebSocket — both sides can talk
- ✓ Server-Sent Events
- ✓ Webhooks — the API that calls you
- ✓ Polling and long-polling
- ✓ How to choose — one table
- ✓ The trade-offs nobody mentions
- ✓ Interview questions

Who it's for

- ✓ Developers who have only ever written REST
- ✓ Backend engineers preparing for interviews
- ✓ Anyone who has picked a style by copying the last project
- ✓ Frontend devs who want to know what the other side does

Why it matters

- ✓ Every style explained with a picture of the traffic
- ✓ Plain language before any jargon
- ✓ Honest trade-offs, not a favourite
- ✓ Ends with interview answers you can say out loud

Comment API and I will DM you the full PDF 



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

1

What an API style actually decides

Six names, three real questions



It is not about the format

Every style below can carry JSON. Arguing about JSON is arguing about the wrapping paper.

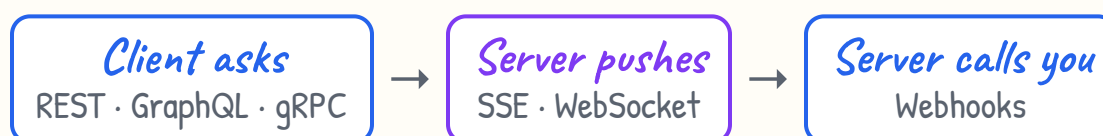
- A style answers who starts the conversation
- And how long the connection stays open
- And who decides the shape of the reply
- Everything else — verbs, status codes — follows from those



The three questions

- Who speaks first — the client, or the server?
- One shot or a stream — does the connection close?
- Who shapes the data — the server, or the caller?
- Answer these three and the style picks itself

Who starts the conversation



the same data can travel any of these ways — the direction is the decision

Key

- ✓ A style is a traffic pattern, not a data format

Takeaways

- ✓ Pick by direction and lifetime of the connection, not by fashion
- ✓ Most real systems use three or four of these at once



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

2

REST — resources and verbs

The default, and the least understood

The idea

Model the system as **resources** — nouns with addresses — and act on them with a small, fixed set of verbs.

- `GET /orders/91` — a thing, at a URL
- The verb says the intent: `GET` `POST` `PUT` `PATCH` `DELETE`
- The status code says what happened: `200` `201` `404` `409`
- You never invent `/getOrderById` — the verb already said it

Stateless is the real rule

- Every request carries **everything** needed to serve it
- The server keeps no memory of your last call
- That is what lets you put **ten servers** behind a load balancer
- It is also why you send the token **every time**

1

Safe & idempotent



- **Safe** — changes nothing: `GET`, `HEAD`
- **Idempotent** — same call twice, same end state: `PUT`, `DELETE`
- `POST` is neither — that is why double-clicks create two orders

2

Caching is the superpower



- A `GET` with a URL can be cached by browsers and **CDNs**
- No other style on this list gets that for free
- This alone keeps REST on the public edge

3

Where it hurts



- One screen often needs **three or four** calls
- Or one call returns far more than the screen uses
- Versioning is manual — `/v1`, `/v2`, forever

4

What it is not



- REST is not "JSON over HTTP"
- Most "REST APIs" are **HTTP APIs** — and that is fine
- The constraints are what buy you caching and scale

Key

- ✓ Resources and verbs, and the server never remembers you

Takeaways

- ✓ Statelessness buys **horizontal scale**; URLs buy **caching**
- ✓ Cost: chatty screens, over-fetching and hand-rolled versioning



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

3

GraphQL — the client picks the fields

One endpoint, and the caller writes the shape



The inversion

REST: the server decides what a response contains.
GraphQL: the client asks for exactly the fields it needs, from one endpoint.

- One URL — usually `POST /graphql`
- A typed **schema** describes everything available
- The query names the fields; the reply mirrors that shape
- Ask for three things at once instead of three round trips



What it kills

- **Over-fetching** — a list screen pulling whole objects
- **Under-fetching** — the call that forces three more
- Version sprawl — you add fields and deprecate old ones
- A mobile app on a bad network feels this immediately

1

The N+1 trap



- One query for 50 orders, then **50 more** for each customer
- The shape is the client's, so the server can't predict it
- Fixed by **batching** resolvers — it is not automatic

2

Caching gets harder



- Everything is a `POST` to one URL
- So HTTP and CDN caching **no longer apply**
- You cache in the client or in the resolver instead

3

New attack surface



- A caller can request a deeply **nested** query
- Depth limits, cost analysis and timeouts are mandatory
- Not optional hardening — the default is exploitable

4

Where it wins



- Many different clients, each needing a different slice
- Fast-moving product teams tired of `/v3`
- Screens that assemble data from several services

Key

- ✓ The client writes the response shape; the server publishes a typed schema

- ✓ **Buyer** fewer round trips, pays with caching, N+1 and query cost control



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

4

gRPC — a contract, not a URL

What services use when no browser is watching



Start from the contract

You write a `.proto` file describing the methods and messages. Both sides generate code from it — there is no hand-written client.

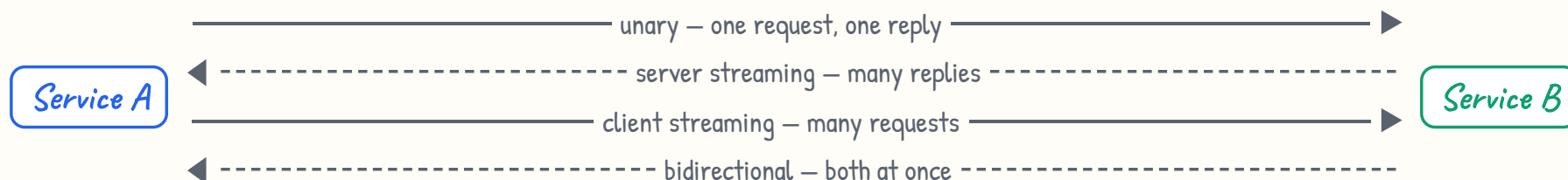
- Calling a remote service looks like calling a function
- Messages travel as Protocol Buffers — binary, not text
- Smaller on the wire and faster to parse than JSON
- Rename a field and the build breaks — before production does



It rides HTTP/2

- Many calls multiplexed over one connection
- Headers compressed, no queue behind a slow response
- That is what makes streaming natural here
- Browsers cannot speak it directly — they need a proxy

The four kinds of gRPC call



four call shapes, all over one HTTP/2 connection

Key

- ✓ A generated client from a shared `.proto` — the contract is the source of truth

Takeaways

- ✓ Binary and multiplexed: the fastest option between your own services
- ✓ Not for public or browser traffic — no URLs, no caching, needs a proxy



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

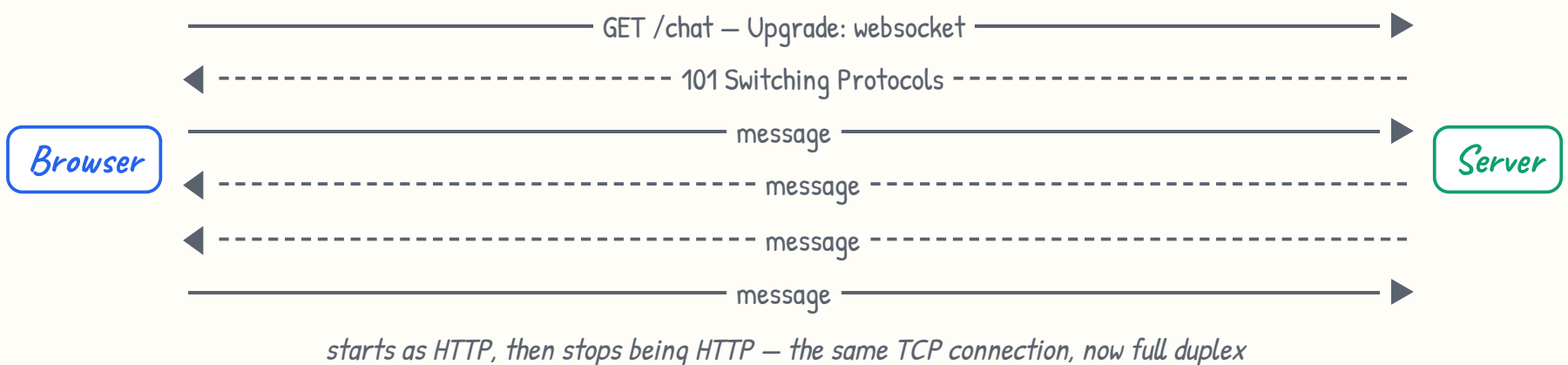
What each one actually is — and when each one wins

5

WebSocket — both sides can talk

One connection that stays open

The upgrade handshake



What actually happens

- A normal HTTP request asks to **upgrade**
- The server answers **101** and the protocol changes
- From then on both sides send **frames** whenever they like
- No request/response pairing — either side may speak first



What it costs

- The server holds **state** per connection — memory per user
- Stateless scaling is gone; you need sticky routing or a bus
- Reconnection, heartbeats and backoff are **your job**
- Some proxies and networks close idle connections

Key

- ✓ HTTP opens the door, then gets out of the way — **full duplex** after **101**

Takeaways

- ✓ The only style here where the **client** can also push at any moment
- ✓ Use it when both directions are genuinely live: chat, multiplayer, collaborative editing



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

6

Server-Sent Events — one way, done simply

The style most teams skip past and should not



A response that never ends

The client makes one ordinary `GET`. The server replies `Content-Type: text/event-stream` and then simply keeps writing.

- Still plain HTTP — proxies, auth and logging all still work
- The browser has it built in: `EventSource`
- It reconnects by itself if the connection drops
- And resumes using the last event id it saw



Why choose it over WebSocket

- Most "real-time" features are server → client only
- Notifications, live prices, progress bars, streamed AI tokens
- No upgrade, no framing, no custom reconnect logic
- Far less to get wrong

1

The limit



- The client cannot send anything on that stream
- It makes a normal request for that — which is usually fine

2

Text only



- The format is UTF-8 text events
- Binary needs encoding, or WebSocket

3

Connection budget



- Over HTTP/1.1 a browser allows only ~6 per domain
- Over HTTP/2 they multiplex and this stops mattering

4

Still a long connection



- Server resources are held open, like WebSocket
- Idle timeouts on proxies still apply

Key

- ✓ One `GET` whose response never closes — server to client only

Takeaways

- ✓ Built into the browser, reconnects and resumes on its own
- ✓ If only the server needs to push, this is the simpler correct answer



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

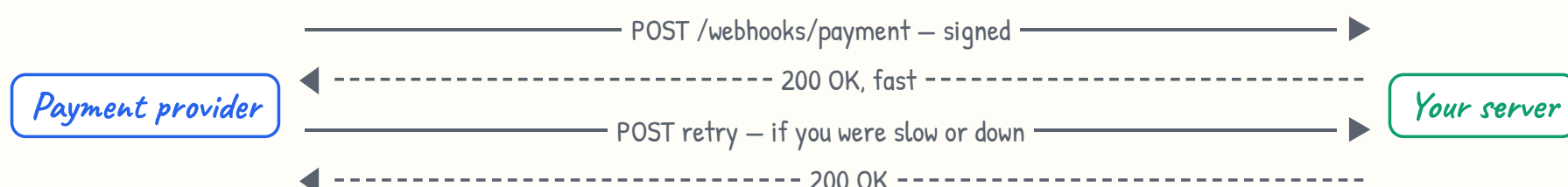
What each one actually is — and when each one wins

7

Webhooks — the API that calls you

Inverted: you publish the endpoint, they call it

A webhook delivery



you are the server now — and the caller will retry until you say 200

1

Why it exists



- Polling "is the payment done yet?" wastes both sides
- Let the event **find you** instead
- Standard for payments, CI, git pushes, provisioning

2

Verify the signature



- The URL is public — **anyone** can POST to it
- Senders sign the body with a shared secret (**HMAC**)
- Unverified webhook = an open door into your system

3

Assume duplicates



- Delivery is **at-least-once**, never exactly-once
- A slow **200** still earns you a retry
- Key each event by id and make handling **idempotent**

4

Answer fast, work later



- Acknowledge in milliseconds, then queue the real work
- Doing the work inline is how you end up with duplicates

Key

- ✓ The direction flips — their server calls yours, on their schedule

Takeaways

- ✓ Always **verify the signature**; the endpoint is public by definition
- ✓ At-least-once means **idempotent handlers**, and a fast 200 before the work



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

8

Polling and long-polling

The baseline everything else is measured against



Short polling

Ask again every few seconds. Simple, and mostly waste — the answer is usually "nothing new".

- Latency is however long you waited
- Load scales with clients × frequency, not with real events
- Still the right answer for genuinely slow-changing data



Long polling

Ask, and the server holds the request open until something happens or it times out.

- Near-real-time without a new protocol
- A held connection per waiting client — same cost as SSE
- The classic fallback before SSE and WebSocket were everywhere

Key Takeaways

- ✓ Polling trades freshness for simplicity — and it is often the right trade
- ✓ Long polling gets the freshness but pays SSE's price without SSE's tooling
- ✓ If you are reaching for long polling today, reach for SSE instead



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

9

How to choose

Three questions, one table



Ask in this order

- 1. Does the **server** need to push?
→ SSE or WebSocket
- 2. Does the **client** also need to push live? → WebSocket
- 3. Is it **service-to-service**, internally? → gRPC
- 4. Many clients, different slices? → GraphQL
- 5. Public, cacheable, plain? → **REST**
- 6. Someone else's event, not yours? → Webhook



What real systems do

- REST at the public edge — caching and tooling
- gRPC between internal services — speed and contracts
- SSE for notifications and streamed output
- WebSocket only where both sides truly talk
- Webhooks for anything a third party owns
- It is **never one style** — it is a stack of them

Key

✓ Direction first, lifetime second, data shape third

Takeaways

✓ The public edge and the internal mesh want **different answers**

✓ "Which is best" has no answer; "which for this traffic" always does



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

10

The trade-offs nobody mentions

What bites six months in

1

Debuggability



- REST: `curl` it, read it, done
- gRPC: binary — you need `grpcurl` and reflection
- The team's **worst night** is a debugging night

2

Caching



- Only REST gets **free** HTTP and CDN caching
- Everything else caches in your own code

3

Versioning



- REST: `/v1`, `/v2`, and a migration each time
- GraphQL and gRPC: **add fields**, deprecate the old ones

4

Load balancing



- Stateless requests balance trivially
- Long-lived connections **pin** a user to one server

5

Who can call it



- Browsers speak REST, GraphQL, SSE, WebSocket
- They cannot speak gRPC without a proxy

6

Failure modes



- Request styles fail **loudly** — you get a status code
- Streams fail **quietly** — it just goes silent
- Heartbeats exist so you notice

Key

- ✓ The style you pick decides how you **debug, cache, version and scale**

Takeaways

- ✓ Long-lived connections trade stateless simplicity for liveness
- ✓ Streams go quiet when they break — **build the heartbeat before you need it**



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

11

Interview questions

The ones that actually get asked

1

REST vs GraphQL?



- "REST fixes the response shape server-side and gets HTTP caching free.
- GraphQL lets the client choose fields, cutting round trips,
- but gives up HTTP caching and invites N+1 and deep-query abuse."

2

Why gRPC internally?



- "A shared .proto generates both sides, so breaking changes fail at build.
- Protobuf over HTTP/2 is smaller and multiplexed.
- Browsers can't speak it — that's why it stays internal."

3

SSE or WebSocket?

- "If only the server pushes, SSE — plain HTTP, built-in reconnect.
- WebSocket when the client must push live too.
- Choosing WebSocket for notifications is over-engineering."

4

Duplicate webhooks?



- "Delivery is at-least-once, so handlers must be idempotent.
- Key on the event id, record processed ids, return 200 fast
- and do the work asynchronously."

5

Is POST idempotent?



- "No — that's why a double-click can create two orders.
- PUT and DELETE are. For POST you add an idempotency key
- the client sends and the server de-duplicates on."

6

Scaling WebSockets?



- "Each connection is state on one server, so you lose stateless
- balancing. You need sticky routing plus a pub/sub bus so any
- server can reach a user held by another."



API Styles Complete Handbook

REST, GraphQL, gRPC, WebSocket, SSE, Webhooks

What each one actually is — and when each one wins

That's every API style, end to end.

What you just went through

- 1 What an API style *actually* decides
- 2 *REST* — resources and verbs
- 3 *GraphQL* — the client picks the fields
- 4 *gRPC* — a contract, not a URL
- 5 *WebSocket* — both sides can talk
- 6 *Server-Sent Events* — one way, done simply
- 7 *Webhooks* — the API that calls you
- 8 *Polling* and long-polling
- 9 How to *choose*
- 10 The *trade-offs* nobody mentions
- 11 *Interview* questions



Save this — you will come back to it



Share it with whoever picks the next API



Comment **API** for the complete PDF in your DM

Written from 10 years of backend engineering — REST, gRPC and streaming APIs in production.

@vikstack